(Phần gạch chân là phần ý chính để vào slide)

Lưu ý trang bìa: ghi tên chủ đề là: Saga Pattern: Orchestration & Choreography

Thành viên: Đặng Quốc Khánh (B22DCCN444) và Nguyễn Thế Lâm (tự ghi MSV vào)

1. Bối cảnh ra đời của Saga Pattern

Slide 1: 1.1 Transation trong kiến trúc Monolithic

- Trong kiến trúc Monolithic, tất cả các logic nghiệp vụ đều nằm trong 1 ứng dụng và dùng chung 1 database duy nhất

- Một transaction trong monolithic luôn đảm bảo ACID: (yêu cầu hình ảnh trong slide)

  + A: Atomicity: tính nguyên tử: một transation gồm các bước nhỏ, là một khối thống nhất không thể tách rời, các bước phải được thực hiện cùng nhau, nếu 1 bước lỗi thì transaction không được thực hiện và dữ liệu trong DB phải nguyên vẹn như khi chưa thực hiện.
  + C: Consistency: tính nhất quán: dữ liệu luôn phải tuân thủ các quy tắc đã quy định trước. Dữ liệu trước và sau thực hiện transaction đều phải hợp lệ theo các quy tắc này. VD: khi chuyển tiền mà sau giao dịch số tiền âm => không hợp lệ, transaction bị hủy
  + I: Isolation: tính cô lập: nhiều transaction chạy song song không làm ảnh hưởng lẫn nhau. Ví dụ: Cùng 1 tài khoản có 2 người rút tiền cùng lúc => người thực hiện trước thì transaction được xử lý trước, người còn lại phải đợi transaction trước xong mới được tiếp tục xử lý
  + D: Durability: tính bền vững: sau khi transaction thành công thì dữ liệu được lưu lại vĩnh viễn vào ổ cứng. Dù có bất kỳ sự cố nào xảy ra thì dữ liệu này vẫn phải bảo đảm nguyên vẹn hoặc có thể backup lại được.

- Transaction trong mô hình monolithic luôn bảo đảm dữ liệu được nhất quán rất mạnh (strong consitency), nếu chỉ có một bước nào đó trong transaction bị lỗi thì tất cả sẽ được rollback tự động

End slide 1

Slide 2: 1.2 Transaction trong kiến trúc microservices

- Trong kiến trúc microservices, mỗi yêu cầu (request) thường đi qua nhiều service khác nhau, mỗi service có DB riêng, nghĩa là request đi đến service A,

service A thực hiện thành công, lưu vào DB của A rồi mới tiếp tục thực hiện service B. (yêu cầu hình ảnh sơ đồ kiểu: service A có DB A, service B có DB B…)

- Tính nguyên tử (atomicity), tính nhất quán (consistency) và tính cô lập (isolation) bị phá vỡ:

+ Tính nguyên tử bị phá vỡ: Khi một hành động thực hiện nhưng gặp lỗi, ví dụ: service A thực hiện thành công => DB của A lưu dữ liệu, nhưng đến service B bị lỗi thì DB của A không tự động rollback lại dữ liệu như ban đầu được.
+ Tính nhất quán bị phá vỡ, nhưng từ nhất quán mạnh trở thành nhất quán cuối cùng: Service A thực hiện xong hành động, lưu vào DB của A, dữ liệu đã thay đổi mặc dù chuỗi transaction chưa thực hiện xong
+ Tính cô lập bị phá vỡ: giữa thời gian các service thực hiện, có khoảng thời gian trống. Người dùng có thể nhìn thấy dữ liệu sau khi service A thành công mặc dù khi đó toàn bộ hành động chưa thực hiện thành công => dữ liệu sẽ bị lệch trong một khoảng thời gian dù rất ngắn

=> Giải pháp:

- Chấp nhận dữ liệu chỉ có sự nhất quán cuối cùng (eventual consistency): dữ liệu có thể bị lệch trong một khoảng thời gian ngắn, nhưng cuối cùng thì dữ liệu vẫn phải chính xác.

- Sử dụng Saga Pattern

End Slide 2

Slide 3+4: 1.3 Saga Pattern

Slide 3

Saga Pattern là một mẫu/ thiết kế dùng để đảm bảo tính nhất quán dữ liệu trong kiến trúc Microservices. Nó chia một transaction lớn thành các transactions nhỏ nối tiếp nhau.

Saga Pattern vận hành theo công thức (P đọc là Process khi thuyết trình) $P_1 \rightarrow P_2 \rightarrow \ldots \rightarrow P_n$, với Pi ($1 < i \leq n$)

Khi xảy ra lỗi, do không thể rollback như monolithic, nó sẽ chạy những transactions theo ý nghĩa bù trừ các transactions đã thực hiện, ví dụ lỗi ở P_k thì nó sẽ chạy $P_k \rightarrow P_{k-1} \rightarrow \ldots P_1$

End slide 3

Slide 4

Ví dụ: Transaction lớn gồm 3 service Order, Inventory và Payment

Luồng bình thường: Order tạo order → Inventory kiểm tra và trừ kho → Payment thanh toán thành công

Luồng lỗi: Order tạo order → Inventory kiểm tra và trừ kho → Payment thanh toán, gặp lỗi → Inventory hoàn lại số lượng của kho → Order cập nhật trạng thái thất bại

End slide 4

2. Orchestration Pattern

Slide 5: Orchestration Pattern

Orchestration Pattern là một dạng mẫu thiết kế phân tán, trong đó có một thành phần điều phối (orchestrator). Nó nắm giữ kịch bản triển khai một hoạt động, bao gồm các bước và thứ tự thực hiện trong luồng hoạt động, cách xử lý bù (compensation) nếu xảy ra lỗi.

Thông thường service khởi tạo quy trình sẽ được chọn làm orchestrator

Thành phần tham gia: Các services. Các services này không biết và cũng không cần biết bản thân đang nằm trong quy trình nào. Chúng chỉ cần nhận lệnh, làm và báo cáo lại kết quả.

Cơ chế: Command based. Orchestrator gửi lệnh đến các service, các service xử lý sau đó gửi reply về orchestrator

End slide 5

Slide 6: Luồng hoạt động bình thường (yêu cầu vẽ workflow trích xuất san ảnh cho đẹp và đủ thông tin để cho vào slide)

Ví dụ luồng hoạt động với 3 service: Order, Inventory và Payment, xét hoạt động tạo đơn hàng và thanh toán. Chọn OrderService là Orchestrator

(ảnh workflow 1)

Bước 1: Client tạo Order → OrderService lưu order mới (POST/order/create) với status là PENDING. Gán transactionId là id cho workflow này

Bước 2: Orchestrator (chính là OrderService) gọi InventoryService (POST/inventory/reserve), truyền theo transactionId và thông tin order, product và số lượng. InventoryService sẽ kiểm tra số lượng và trừ đi trong kho (nếu đủ), gửi lại phản hồi cho Orchestrator

Bước 3: Orchestrator nhận phản hồi từ InventoryService, sau đó Orchestrator gọi PaymentService (POST/payment/charge), truyền theo transactionId, thông tin order và thanh toán, nếu thành công, thực hiện lưu payment record vào DB, sau đó reply orchestrator trạng thái payment

Bước 4: Orchestrator nhận phản hồi từ PaymentService, sau đó sửa trạng thái của order từ PENDING thành DONE (Orchestrator không cần gọi đến OrderService do OrderService chính là Orchestrator). Truyền theo paymentId, transactionId và orderId

Hết Slide 6

Slide 7: Luồng hoạt động lỗi

(ảnh workflow 2) Lỗi ở bước kiểm tra kho

Bước 1, 2 tương tự như quy trình chuẩn

Bước 3: Orchestrator nhận phản hồi từ InventoryService sau đó sửa trạng thái PENDING thành FAILED. Truyền theo transactionId và orderId

(ảnh workflow 3, có thể để ở slide thứ 8) Lỗi ở bước Payment

Bước 1,2,3 tương tự như quy trình chuẩn

Bước 4: Orchestrator nhận phản hồi từ PaymentService, sau đó gọi InventoryService (POST/inventory/refund). Truyền theo transactionId và orderId để tìm ra lần trừ kho tương ứng, sau đó trả lại số lượng về kho theo số lượng đã trừ trương ứng rồi gửi reply đến Orchestrator

Bước 5: Orchestrator nhận reply từ InventoryService, sau đó cập nhật trạng thái order từ PENDING thành FAILED

Hết slide 7

Một số lưu ý khi triển khai

Orchestration cần lưu trạng thái saga (id, bước/trạng thái hiện tại) để có thể resume khi gặp lỗi và truy vết lịch sử hành động, đồng thời kiểm tra transactionId khi thực hiện ở mỗi service để tránh trùng lặp

Phân biệt lỗi cần rollback/retry:

+ Retry thực hiện với các request gặp lỗi tạm thời như lỗi về mạng hoặc service unavailable. Nếu sau nhiều lần retry không được thì phải lưu lại các request này vào 1 file log.
+ Với những lỗi nằm trong luồng nghiệp vụ, ví dụ như refund inventory nếu payment lỗi thì việc rollback là cần thiết, nhưng với những lỗi như khi gửi thông báo cho người dùng về việc order thành công hay thất bại thì đây là lỗi không cần rollback, chỉ cần ghi lại lỗi và retry.

Slide 8: Ưu và nhược điểm (cho hết vào slide)

Ưu điểm

+ Luồng nghiệp vụ được kiểm soát tập trung tại orchestrator => dễ quan sát, triển khai, kiểm tra, truy vấn và debug
+ Phù hợp với những nghiệp vụ phức tạp, cần kiểm soát chặt chẽ các điều kiện

Nhược điểm

+ Biến orchestrator thành điểm tập trung xử lý logic, làm nó trở nên phức tạp khi số services tăng cao hoặc luồng nghiệp vụ dài
+ Gây ra sự phụ thuộc (coupling) giữa orchestrator và các service

3. Choreography Pattern

Slide 9: Chorepgraphy Pattern

<u>Choreography Pattern là một mẫu thiết kế phân tán không có orchestrator trung tâm.</u>

<u>Cơ chế: Event based:</u> dựa trên sự kiện. Các services giao tiếp với nhau thông qua events qua message broker như Kafka hay RabbitMQ. Trong message broker, mỗi serivce sẽ có một topic riêng để publish (gửi) sự kiện lên. Các services cũng lắng nghe sự kiện từ những topics của các services khác. Mỗi service publish event khi hoàn thành nhiệm vụ, các service khác subscribe và tiếp tục workflow.

Slide 10 + 11: Luồng hoạt động bình thường

Ví dụ luồng hoạt động với 3 service: Order, Inventory và Payment, xét hoạt động tạo đơn hàng và thanh toán. order-service publish gói tin lên kênh

order-events, subcribe kênh inventory-events và payment-events để lắng nghe; inventory pulish gói tin lên kênh inventory-events và lắng nghe ở 2 kênh order-events và payment-events; payment-service publish lên 2 kênh payment-events và lắng nghe ở kênh inventory-events. (nên tạo thêm 1 slide vẽ 1 hình ảnh chỉ rõ mối quan hệ giữa các service và topics. Các topics nằm trong message broker Kafka và vẽ thêm các DB tương ứng với từng service)

Bước 1: Khách hàng gọi API POST /orders. OrderService tiếp nhận, lưu đơn hàng vào Database với trạng thái PENDING và sinh ra một transactionId. OrderService đóng gói một gói tin có tên là OrderCreatedEvent (Sự kiện Đơn hàng đã được tạo), đính kèm transactionId và danh sách món hàng. Nó ném sự kiện này lên Topic order-events. Lúc này, OrderService đi làm việc khác, không chờ đợi ai cả.

Bước 2: InventoryService vốn đang lắng nghe ở topic order-events. Nó bắt được gói tin OrderCreatedEvent. Nó sẽ đọc data, chui vào Database của nó để trừ số lượng sản phẩm. Trừ kho thành công, InventoryService ném một gói tin InventoryReservedEvent (Sự kiện Kho đã giữ hàng) lên topic inventory-events.

Bước 3: PaymentService đang theo dõi Topic inventory-events. Nó tóm được gói tin InventoryReservedEvent. Nó thực hiện thao tác thanh toán. Trừ tiền thành công, nó ném gói tin PaymentBilledEvent (Sự kiện Đã thu tiền) lên topic payment-events.

Bước 4: OrderService vẫn luôn lắng nghe ở Topic payment-events. Nó bắt được sự kiện PaymentBilledEvent. Nó truy vấn DB của nó, tìm đúng đơn và cập nhật trạng thái từ PENDING sang COMPLETED

Slide 12 + 13: Luồng hoạt động lỗi

Bước 1 & Bước 2: Diễn ra bình thường như trên. Kho đã giữ hàng và bắn ra InventoryReservedEvent.

Bước 3: Sự cố xảy ra: PaymentService tóm được event giữ hàng, tiến hành trừ tiền. Nhưng cổng thanh toán báo "Không đủ số dư".Lúc natf PaymentService ném ra một gói tin PaymentFailedEvent (Sự kiện Thanh toán thất bại) lên Topic payment-events.

Bước 4: Tại InventoryService đang lắng nghe Topic payment-events tóm được PaymentFailedEvent. Nó tra mã đơn hàng, lôi hàng đã giữ ra cộng ngược lại vào kho

Tại OrderService bắt được PaymentFailedEvent. Nó tra DB, đổi đơn tương ứng từ PENDING sang FAILED.

Slide 14: Ưu – nhược điểm

Ưu điểm

+ Không có single point of orchestration, tránh được việc service bị sập, giống như việc điều tiết giao thông ở ngã tư, khi có cảnh sát thực hiện thì hoạt động mượt mà, nhưng khi cảnh sát không thực hiện được nhiệm vụ đó nữa thì sẽ gây ra tê liệt ách tắc giao thông. Với choreograpy, nó giống như 4 đèn giao thông ở 4 phía, các màu đèn giống như các gói tin để con người biết nên đi hay không.
+ Loose coupling tối đa giữa các services. Service này không cần biết có service kia hay không mà chỉ cần bắt gói tin thông qua message broker
+ Có thể mở rộng (Scale) từng service độc lập
+ Phản ứng nhanh với events

Nhược điểm

+ Khó debug và test (do events phân tán, không có nơi nào lưu lại toàn bộ vòng đời của transaction từ A-Z)
+ Phức tạp trong việc đảm bảo thứ tự sự kiện, ví dụ: luồng hoạt động ở trên bị lỗi ở chỗ Payment, thì kho vừa hoàn hàng, order vừa cập nhật thành FAILED, phải bảo đảm được kho hoàn hàng xong thì order mới FAILED
+ Cần cơ sở hạ tầng công nghệ mạnh (như message broker, hay hệ thống giám sát)

Slide 15: So sánh orchestration và choreography

Tiêu chí	Saga Orchestration	Saga Choreography
Cơ chế hoạt động	Command driven	Event driven
Điều khiển	Thông qua một orchestrator để điều phối	Phân tán, không có người điều phối
Giao tiếp	Thường dùng lệnh đồng bộ như RESTful API / gRPC	Bất đồng bộ thông qua message broker
Tính độc lập	Trung bình, orchestrator phải biết được tên, địa chỉ của dịch vụ khác	Không phụ thuộc nhau
Theo dõi tiến độ	Ngay trong log/CSDL của orchestrator	Phải truy vết theo từng service

Khả năng mở rộng	Trung bình, do phải sửa code trong orchestrator	Dễ dàng do chỉ cần lắng nghe sự kiện chứ không phụ thuộc vào service khác

4. So sánh và lựa chọn các mẫu thiết kế

- Với Monolithic: có độ phức tạp thấp, dễ debug, tuy nhiên khó mở rộng và sự phụ thuộc cao giữa các module, do đó sử dụng với hệ thống nhỏ, không quá nhiều người dùng, yêu cầu tốc độ phát triển nhanh, không có nhiều nhu cầu mở rộng (scale) phức tạp

- Với Orchestration: độ phức tạp trung bình, khả năng debug và scale trung bình, độ phụ thuộc giữa các module trung bình, do đó có thể sử dụng khi cần kiểm soát luồng nghiệp vụ rõ ràng, khi logic xử lý nghiệp vụ phức tạp với nhiều điều kiện phân nhánh

- Với Choreography: độ phức tạp cao, khó debug, dễ mở rộng, độ phụ thuộc giữa các module thấp, do đó phù hợp với hệ thống cần giảm sự phụ thuộc tối đa, cần scale mở rộng độc lập từng service.

An sao trong tử vi

02/07/2004 âm giờ Mùi

An 12 giáp: Tý – Sửu - … - Hợi theo thứ tự 12 con giáp và cố định trong lá số như dưới đây

Cự Môn QUAN LỘC Tỵ	Thiên Tướng Liêm Trinh NÔ BỘC Ngọ	Thiên Lương THIÊN DI Mùi	Thất Sát TẬT ÁCH Thân

Tham Lang ĐIỀN TRẠCH Thìn			Thiên Đồng TÀI BẠCH Dậu
Thái Âm PHÚC <THÂN> Mão			Vũ Khúc TỬ TỨC Tuất
Tử Vi Thiên Phủ PHỤ MẪU Dần	Thiên Cơ MỆNH Sửu	Phá Quân HUYNH ĐỆ Tý	Thái Dương PHU THÊ Hợi

1. An cung MỆNH và THÂN

Giả sử: Sinh ngày 02/07/2004 ÂM LỊCH giờ Mùi

Bắt đầu tại DẦN, đếm theo tháng sinh (tháng 7): Dần 1 , Mão 2,…. Thân 7. Lấy gốc tại Thân. An Mệnh: từ gốc thân, đếm ngược chiều kim đồng hồ sao cho đến giờ sinh: Thân – Tý, Mùi – Sửu,….. Sửu – Mùi => Mệnh an tại Sửu. An THÂN đếm theo chiều kim đồng hồ tương tự: Thân – Tý, Dậu – Sửu,… Mão – Mùi => Thân an tại Mão

2. An các cung còn lại: Theo chiều kim đồng hồ:

Mệnh → Phụ Mẫu → Phúc Đức → Điền Trạch → Quan Lộc → Nô Bộc → Thiên Di → Tật Ách → Tài Bạch → Tử Tức → Phu Thê → Huynh Đệ

Quy tắc nhớ:

Theo chiều kim đồng hồ

Mệnh là bản thân mình

Phụ Mẫu sinh ra mình nên đứng trước mệnh

Phúc Đức là ông bà tổ tiên sinh ra Phụ Mẫu

Điền trạch là đất đai nơi ông bà tổ tiên ở

Quan Lộc là điều kiện để tạo ra cơ ngơi, đất đai

Nô Bộc là khi thăng quan tiến chức, làm công danh ắt phải có người xung quanh

Thiên Di là muốn gặp những người xung quanh thì phải đi ra xã hội

Tật Ách xảy ra khi chúng ta bôn ba ngoài xã hội nhiều

Tài Bạch là tiền tài ắt thu được sau khi trải qua được bệnh tật, ách nạn

Xét về phía ngược lại chiều kim đồng hồ

Trong gia đình 1 bên là cha mẹ 1 bên là Huynh đệ

Sau này khi lớn, Huynh đệ tách nhau ra thì chúng ta sống với Phu Thê

Tử tức là con cái sinh ra sau khi cưới vợ/chồng

Hoặc có thể nhớ theo vị trí xung đối

Mệnh – Thiên Di: mệnh là con người thật bên trong, Di là con người biểu hiện ra ngoài xã hội

Phụ Mẫu – Tật Ách: cha mẹ sinh ra mình, sức khỏe bệnh tật có tính di truyền từ cha mẹ sang con cái

Phúc Đức – Tài Bạch: có Phúc (phúc đức) mới sinh ra Phần (tiền tài)

Điền Trạch – Tử Tức: cả đời làm việc mua nhà mua đất cũng để cho con cái

Quan Lộc – Phu Thê: Phía sau công danh sự nghiệp của một người luôn có bóng dáng của người bạn đời

Nô bộc – Huynh Đệ: Anh em trong nhà vs Anh em xã hội

3. An chính tinh

3.1 An sao Tử Vi

Tính: Ngày sinh Âm Lịch / Cục, nếu lẻ làm tròn lên. (cục xem sách để biết bản thân thuộc gì)

Ví dụ sinh 2004 => Thủy

Danh sách cục: thủy = 2; mộc = 3; kim = 4; thổ = 5, hỏa = 6.

Ví dụ sinh ngày 2 âm, mệnh Thủy => tính 2/2 = 1

Ví dụ sinh ngày 10 âm, mệnh mộc => tính 10/3 = 3.333 làm tròn lên 4

An sao Tử Vi: bắt đầu tại Dần là 1, đếm đến số vừa tính sẽ xác định được tử vi ở đâu.

Ví dụ: 1 => Tử Vi tại Dần

Ví dụ: 4 => Tử Vi cư Tỵ

3.2 An các chính tinh còn lại

Để an được các chính tinh còn lại, cần biết khái niệm xung đối, nhị hợp, lục xung và tam hợp (tất cả tổ hợp đều xếp theo chiều thuận kim đồng hồ)

a) Tam hợp: 1 cung sẽ tam hợp với các cung cách nó 4 cung, nhìn lên lá số => dễ thấy Dần Ngọ Tuất, Mão Mùi Hợi, Thân Tý Thìn, Tỵ Dậu Sửu

b) Xung đối: Là các cung đối xứng qua trung tâm lá số: Ví dụ: Tý Ngọ, Sửu Mùi, Tỵ Hợi, Dần Thân, Mão Dậu, Thìn Tuất

c) Nhị hợp: đối xứng theo chiều ngang trên lá số: ví dụ: Thìn Dậu, Mão Tuất,…

d) Lục Xung: 2 cung đối xứng theo chiều dọc: ví dụ: Ngọ Sửu, Mùi Tý

Dựa vào vị trí sao tử vi, an các sao như sau:

1. Cự Môn: tử vi là ông vua => rất sợ thị phi, quan nịnh thần => Lục xung cự môn

2. Thiên Cơ: Phía sau Tử Vi luôn là Thiên Cơ tư vấn chuyên môn

3+4. Liêm Trinh, Vũ Khúc: Vòng Vũ Tử Liêm (vũ khúc – tử vi – liêm trinh) (theo chiều kim đồng hồ) thuộc tam hợp, an theo tam hợp

5. Thiên Tướng: là quan võ, lục xung với Thiên Cơ: Quan võ luôn không hợp quan văn

6. Phá quân: luôn xung đối với thiên tướng (đồng thời nhị hợp với Thiên Cơ, phá quân là ông vua thời kỳ trước, nên thiên cơ sẽ chơi với ông vua này)

7+8. Thất Sát, Tham Lang an theo vòng Sát Phá Tham (thất sát – phá quân – tham lang) thuộc tam hợp

9. Thái Âm là người thích tiền => hợp với Vũ Khúc là người giàu, nhiều tiền nên nhị hợp

10. Thiên Lương: là người lương thiện => Nhị hợp với liêm trinh là tượng trưng cho pháp luật

11. Thái Dương: Thái Dương là người giỏi, người giỏi ganh đua đố kỵ với người giỏi khác là Thất Sát nên 2 cung này lục xung

12. Thiên Phủ: người giỏi có công danh là Thái Dương nên cũng có cơ ngơi, phủ, do đó Thái Dương nhị hợp Thiên Phủ

13. Thiên Đồng: nhị hợp với Tham Lang do 2 người này đều mê tín bậc nhất trong khoa Tử Vi